

LIST IN PYTHON

Introduction:

- List is a collection of elements which is ordered and changeable (mutable).
- Allows duplicate values.
- A list contains items separated by commas and enclosed within square brackets ([]).
- All items belonging to a list can be of different data type.
- The values stored in a list can be accessed using the slice operator ([] and [:]) with indexes starting at 0 in the beginning of the list.

Difference between list and string:

List	String
Mutable	Immutable
Element can be assigned at specified index	Element/character cannot be assigned at specified index.
Example: >>>L=[7,4,8,9] >>>L[2]=6 #valid	Example: >>>str= "python" >>>str[2]= 'p' #error

Creating a list:

To create a list enclose the elements of the list within square brackets and separate the elements by commas.

Syntax:

list-name= [item-1, item-2,, item-n]

Example:

```
mylist = ["apple", "banana", "cherry"]       # a list with three items
```

```
L = [ ]               # an empty list
```

Creating a list using list() Constructor:

o It is also possible to use the list() constructor to make a list.

```
mylist = list(("apple", "banana", "cherry"))       #note the double round-brackets print(mylist)
```

```
L=list( )           # creating empty list
```

Nested Lists:

```
>>> L=[23,'w',78.2, [2,4,7],[8,16]]
```

```
>>> L
```

```
[23, 'w', 78.2, [2, 4, 7], [8, 16]]
```

Creating a list by taking input from the user:

```
>>> List=list(input("enter the elements: "))
```

```
enter the elements: hello python
```

```
>>> List
```

```
['h', 'e', 'l', 'l', 'o', ' ', 'p', 'y', 't', 'h', 'o', 'n']
```

```
>>> L1=list(input("enter the elements: "))
```

```
enter the elements: 678546
```

```
>>> L1
```

```
['6', '7', '8', '5', '4', '6']      # it treats elements as the characters though we entered digits
```

To overcome the above problem, we can use eval() method, which identifies the data type and evaluate them automatically.

```
>>> L1=eval(input("enter the elements: "))
```

```
enter the elements: 654786
```

```
>>> L1
```

```
654786      # it is an integer, not a list
```

```
>>> L2=eval(input("enter the elements: "))
```

```
enter the elements: [6,7,8,5,4,3]      # for list, you must enter the [ ] bracket
```

```
>>> L2
```

```
[6, 7, 8, 5, 4, 3]
```

Note: With eval() method, If you enter elements without square bracket[], it will be considered as a tuple.

```
>>> L1=eval(input("enter the elements: "))
```

enter the elements: 7,65,89,6,3,4

```
>>> L1
```

```
(7, 65, 89, 6, 3, 4)    #tuple
```

Accessing lists:

- The values stored in a list can be accessed using the slice operator ([] and [:]) with indexes.
- **List-name[start:end]** will give you elements between indices **start** to **end-1**.
- The first item in the list has the index zero (0).

12	56	87	45	23	97	56	27
----	----	----	----	----	----	----	----

Example:

```
>>> number=[12,56,87,45,23,97,56,27]
```

Forward Index							
0	1	2	3	4	5	6	7
-8	-7	-6	-5	-4	-3	-2	-1

Backward Index

```
>>> number[2]
```

```
87
```

```
>>> number[-1]
```

```
27
```

```
>>> number[-8]
```

```
12
```

```
>>> number[8]
```

```
IndexError: list index out of range
```

```
>>> number[5]=55          #Assigning a value at the specified index
```

```
>>> number
```

```
[12, 56, 87, 45, 23, 55, 56, 27]
```

Traversing a LIST:

Traversing means accessing and processing each element.

Method-1:

```
>>> day=list(input("Enter elements :"))
```

```
Enter elements : sunday
```

```
>>> for d in day:
```

```
print(d)
```

Output:

s
u
n
d
a
y

Method-2

```
>>> day=list(input("Enter elements :"))  
Enter elements : wednesday  
>>> for i in range(len(day)):  
    print(day[i])
```

Output:

w
e
d
n
e
s
d
a
y

7.5 List Operators:

- Joining operator +
- Repetition operator *
- Slice operator [:]
- Comparison Operator <, <=, >, >=, ==, !=

□ **Joining Operator:** It joins two or more lists.

Example:

```
>>> L1=['a',56,7.8]
```

```
>>> L2=['b','&',6]
```

```
>>> L3=[67,'f','p']
```

```
>>> L1+L2+L3
```

```
['a', 56, 7.8, 'b', '&', 6, 67, 'f', 'p']
```

▯ **Repetition Operator:** It replicates a list specified number of times.

Example:

```
>>> L1*3
```

```
['a', 56, 7.8, 'a', 56, 7.8, 'a', 56, 7.8]
```

```
>>> 3*L1
```

```
['a', 56, 7.8, 'a', 56, 7.8, 'a', 56, 7.8]
```

▯ **Slice Operator:**

List-name[start:end] will give you elements between indices **start** to **end-1**.

```
>>> number=[12,56,87,45,23,97,56,27]
```

```
>>> number[2:-2] [87, 45,  
23, 97]
```

```
>>> number[4:20] [23, 97,  
56, 27]
```

```
>>> number[-1:-6] [ ]
```

```
>>> number[-6:-1] [87, 45,  
23, 97, 56]
```

```
>>> number[0:len(number)] [12, 56, 87,  
45, 23, 97, 56, 27]
```

List-name[start:end:step] will give you elements between indices **start** to **end-1** with skipping elements as per the value of **step**.

```
>>> number[1:6:2] [56, 45,  
97]
```

```
>>> number[: :-1]
```

```
[27, 56, 97, 23, 45, 87, 56, 12]      #reverses the list
```

List modification using slice operator:

```
>>> number=[12,56,87,45,23,97,56,27]
```

```
>>> number[2:4]=["hello","python"]
```

```
>>> number
```

```
[12, 56, 'hello', 'python', 23, 97, 56, 27]
```

```
>>> number[2:4]=["computer"]
```

```
>>> number
```

```
[12, 56, 'computer', 23, 97, 56, 27]
```

Note: The values being assigned must be a sequence (list, tuple or string)

Example:

```
>>> number=[12,56,87,45,23,97,56,27]
```

```
>>> number=[12,56,87,45,23,97,56,27]
```

```
>>> number[2:3]=78 # 78 is a number, not a sequence
```

TypeError: can only assign an iterable

□ **Comparison Operators:**

- o Compares two lists
- o Python internally compares individual elements of lists in lexicographical order.
- o It compares the each corresponding element must compare equal and two sequences must be of the same type.
- o For **non-equal comparison** as soon as it gets a result in terms of True/False, from corresponding elements' comparison. If Corresponding elements are equal, it goes to the next element and so on, until it finds elements that differ.

Example:

```
>>>L1, L2 = [7, 6, 9], [7, 6, 9]
```

```
>>>L3 = [7, [6, 9] ]
```

For Equal Comparison:

Comparison	Result	Reason
>>>L1==L2	True	Corresponding elements have same value and same type
>>>L1==L3	False	Corresponding values are not same

For Non-equal comparison:

Comparison	Result	Reason
>>> L1>L2	False	All elements are equal

>>> L2>L3	TypeError: '>' not supported between instances of 'int' and 'list'	in L2, element at the index 1 is int type and in L3 element at the index 1 is list type
>>>[3,4,7,8]<[5,1]	True	3<5 is True
>>>[3,4,7,8]<[3,4,9,2]	True	First two elements are same so move to next element and 7<9 is True
>>>[3,4,7,8]<[3,4,9,11]	True	7<9 is True
>>>[3,4,7,8]<[3,4,7,5]	False	8<5 is False

List Methods:

Consider a list:

```
company=["IBM","HCL","Wipro"]
```

S. No.	Function Name	Description	Example
1	append()	To add element to the list at the end. Syntax: list-name.append (element)	>>> company.append("Google") >>> company ['IBM', 'HCL', 'Wipro', 'Google']
Error: >>>company.append("infosys","microsoft") # takes exactly one element TypeError: append() takes exactly one argument (2 given)			
2	extend()	Add a list, to the end of the current list. Syntax: list-name.extend(list)	>>>company=["IBM","HCL","Wipro"] >>> desktop=["dell","HP"] >>> company.extend(desktop) >>> company ['IBM', 'HCL', 'Wipro', 'dell', 'HP']
Error: >>>company.extend("dell","HP") #takes only a list as argument TypeError: extend() takes exactly one argument (2 given)			
3.	len()	Find the length of the list. Syntax: len(list-name)	>>>company=["IBM","HCL","Wipro"] >>> len(company) 3 >>> L=[3,6,[5,4]] >>> len(L) 3

4	index()	Returns the index of the first element with the specified value. Syntax: list-name.index(element)	>>> company = ["IBM", "HCL", "Wipro", "HCL","Wipro"] >>> company.index("Wipro") 2
Error: >>> company.index("WIPRO") # Python is case-sensitive language ValueError: 'WIPRO' is not in list			
>>> company.index(2) # Write the element, not index ValueError: 2 is not in list			
5	insert()	Adds an element at the specified position. Syntax: list.insert(index, element)	>>>company=["IBM","HCL","Wipro"] >>> company.insert(2,"Apple") >>> company ['IBM', 'HCL', 'Apple', 'Wipro'] >>> company.insert(16,"Microsoft") >>> company ['IBM', 'HCL', 'Apple', 'Wipro', 'Microsoft'] >>> company.insert(-16,"TCS") >>> company ['TCS', 'IBM', 'HCL', 'Apple', 'Wipro', 'Microsoft']
6	count()	Return the number of times the value appears. Syntax: list-name.count(element)	>>> company = ["IBM", "HCL", "Wipro", "HCL","Wipro"] >>> company.count("HCL") 2 >>> company.count("TCS") 0
7	remove()	To remove an element from the list. Syntax: list-name.remove(element)	>>> company = ["IBM", "HCL", "Wipro", "HCL","Wipro"] >>> company.remove("Wipro") >>> company ['IBM', 'HCL', 'HCL', 'Wipro']
Error: >>> company.remove("Yahoo") ValueError: list.remove(x): x not in list			

8	clear()	Removes all the elements from list. Syntax: list-name.clear()	>>> company=["IBM","HCL", "Wipro"] >>> company.clear() >>> company []
9	pop()	Removes the element at the specified position and returns the deleted element. Syntax: list-name.pop(index) The index argument is optional. If no index is specified, pop() removes and returns the last item in the list.	>>>company=["IBM","HCL", "Wipro"] >>> company.pop(1) 'HCL' >>> company ['IBM', 'Wipro'] >>> company.pop() 'Wipro'
Error: >>>L=[] >>>L.pop() IndexError: pop from empty list			
10	copy()	Returns a copy of the list. Syntax: list-name.copy()	>>>company=["IBM","HCL", "Wipro"] >>> L=company.copy() >>> L ['IBM', 'HCL', 'Wipro']
11	reverse()	Reverses the order of the list. Syntax: list-name.reverse() Takes no argument, returns no list.	>>>company=["IBM","HCL", "Wipro"] >>> company.reverse() >>> company ['Wipro', 'HCL', 'IBM']

12.	sort()	Sorts the list. By default in ascending order. Syntax: list-name.sort()	<pre>>>>company=["IBM","HCL", "Wipro"] >>>company.sort() >>> company ['HCL', 'IBM', 'Wipro']</pre> To sort a list in descending order: <pre>>>>company=["IBM","HCL", "Wipro"] >>> company.sort(reverse=True) >>> company ['Wipro', 'IBM', 'HCL']</pre>
-----	---------	---	---

Deleting the elements from the list using *del* statement: Syntax:

del list-name[index] # to remove element at specified index
del list-name[start:end] # to remove elements in list slice

Example:

```
>>> L=[10,20,30,40,50]

>>> del L[2]            # delete the element at the index 2

>>> L

[10, 20, 40, 50]

>>> L= [10,20,30,40,50]

>>> del L[1:3]            # deletes elements of list from index 1 to 2.

>>> L

[10, 40, 50]

>>> del L            # deletes all elements and the list object too.

>>> L
```

NameError: name 'L' is not defined

Difference between del, remove(), pop(), clear():

S. No.	del	remove()	pop()	clear()
1	Statement	Function	Function	Function
2	Deletes a single element or a list slice or complete list.	Removes the first matching item from the list.	Removes an individual item and returns it.	Removes all the elements from list.
3	Removes all elements and deletes list object too.			Removes all elements but list object still exists.

Difference between append(), extend() and insert() :

S. No.	append()	extend()	insert()
1	Adds single element in the end of the list.	Add a list in the end of the another list	Adds an element at the specified position. (Anywhere in the list)
2	Takes one element as argument	Takes one list as argument	Takes two arguments, position and element.
3	The length of the list will increase by 1.	The length of the list will increase by the length of inserted list.	The length of the list will increase by 1.

ACCESSING ELEMENTS OF NESTED LISTS:

Example:

```
>>> L=["Python", "is", "a", ["modern", "programming"], "language", "that", "we", "use"]
```

```
>>> L[0][0]
```

```
'P'
```

```
>>> L[3][0][2]
```

```
'd'
```

```
>>> L[3:4][0]
```

```
['modern', 'programming']
```

```
>>> L[3:4][0][1]
```

```
'programming'
```

```
>>> L[3:4][0][1][3]
```

```
'g'
```

```
>>> L[0:9][0]
```

```
'Python'
```

```
>>> L[0:9][0][3]
```

```
'h'
```

```
>>> L[3:4][1]
```

```
IndexError: list index out of range
```

TUPLE IN PYTHON

INTRODUCTION:

- Tuple is a collection of elements which is **ordered and unchangeable (Immutable)**.
- Immutable means you cannot change elements of a tuple in place.
- Allows duplicate members.
- Consists the values of any type, separated by comma.
- Tuples are enclosed within parentheses ().
- Cannot remove the element from a tuple.

Creating Tuple:

Syntax:

```
tuple-name = ( )    # empty tuple
```

```
tuple-name = (value-1, value-2, ..... , value-n)
```

Example:

```
>>> T=(23, 7.8, 64.6, 'h', 'say')
```

```
>>> T
```

```
(23, 7.8, 64.6, 'h', 'say')
```

Creating a tuple with single element:

```
>>> T=(3)    #With a single element without comma, it is a value only, not a tuple
```

```
>>> T
```

```
3
```

```
>>> T= (3, )  # to construct a tuple, add a comma after the single element
```

```
>>> T
```

```
(3,)
```

```
>>> T1=3,     # It also creates a tuple with single element
```

```
>>> T1 (3,)
```

Creating a tuple using tuple() constructor:

It is also possible to use the **tuple()** constructor to create a tuple.

```
>>>T=tuple( )           # empty tuple
```

```
>>> T=tuple((45,3.9, 'k',22))    #note the double round-brackets
>>> T
(45, 3.9, 'k', 22)
```

```
>>> T2=tuple('hello') # for single round-bracket, the argument must be of sequence type
>>> T2
('h', 'e', 'l', 'l', 'o')
```

```
>>> T3=('hello','python')
>>> T3
('hello', 'python')
```

Nested Tuples:

```
>>> T=(5,10,(4,8))
>>> T
(5, 10, (4, 8))
```

Creating a tuple by taking input from the user:

```
>>> T=tuple(input("enter the elements: "))
```

enter the elements: hello python

```
>>> T
('h', 'e', 'l', 'l', 'o', ' ', 'p', 'y', 't', 'h', 'o', 'n')
```

```
>>> T1=tuple(input("enter the elements: "))
```

enter the elements: 45678

```
>>> T1
```

```
('4', '5', '6', '7', '8')    # it treats elements as the characters though we entered digits
```

To overcome the above problem, we can use eval() method, which identifies the data type and evaluate them automatically.

```
>>> T1=eval(input("enter the elements: "))
```

```
enter the elements: 56789
```

```
>>> T1
```

```
56789      # it is not a list, it is an integer value
```

```
>>> type(T1)
```

```
<class 'int'>
```

```
>>> T2=eval(input("enter the elements: "))
```

```
enter the elements: (1,2,3,4,5)          # Parenthesis is optional
```

```
>>> T2
```

```
(1, 2, 3, 4, 5)
```

```
>>> T3=eval(input("enter the elements: "))
```

```
enter the elements: 6, 7, 3, 23, [45,11]    # list as an element of tuple
```

```
>>> T3
```

```
(6, 7, 3, 23, [45, 11])
```

Accessing Tuples:

Tuples are very much similar to lists. Like lists, tuple elements are also indexed. Forward indexing as 0,1,2,3,4..... and backward indexing as -1,-2,-3,-4,.....

- The values stored in a tuple can be accessed using the slice operator ([] and [:]) with indexes.
- **tuple-name[start:end]** will give you elements between indices **start** to **end-1**.
- The first item in the tuple has the index zero (0).

Example:

```
>>> alpha=('q','w','e','r','t','y')
```

Forward Index					
0	1	2	3	4	5
q	w	e	r	t	y
-6	-5	-4	-3	-2	-1
Backward Index					

```
>>> alpha[5]
```

```
'y'
```

```
>>> alpha[-4]
```

```
'e'
```

```
>>> alpha[46]
```

IndexError: tuple index out of range

```
>>> alpha[2]='b'    #can't change value in tuple, the value will remain unchanged
```

TypeError: 'tuple' object does not support item assignment

Difference between List and Tuple:

S. No.	List	Tuple
1	Ordered and changeable (Mutable)	Ordered but unchangeable (Immutable)
2	Lists are enclosed in brackets. []	Tuples are enclosed in parentheses. ()
3	Element can be removed.	Element can't be removed.

Traversing a Tuple: Syntax:

```
for <variable> in tuple-name:
```

```
    statement
```

Example:

Method-1

```
>>> alpha=('q','w','e','r','t','y')
```

```
>>> for i in alpha:
```

```
    print(i)
```

Output:

```
q
```

```
w
```

```
e
```

```
r
```

```
t
```

```
y
```

Method-2

```
>>> for i in range(0, len(alpha)):
```

```
    print(alpha[i])
```


Output:

q
w
e
r
t
y

Tuple Operations:

- Joining operator +
- Repetition operator *
- Slice operator [:]
- Comparison Operator <, <=, >, >=, ==, !=

- **Joining Operator:** It joins two or more tuples.

Example:

```
>>> T1 = (25,50,75)
>>> T2 = (5,10,15)
>>> T1+T2
(25, 50, 75, 5, 10, 15)
```

```
>>> T1 + (34)
TypeError: can only concatenate tuple (not "int") to tuple
```

```
>>> T1 + (34, )
(25, 50, 75, 34)
```

- **Repetition Operator:** It replicates a tuple, specified number of times.

Example:

```
>>> T1*2
(25, 50, 75, 25, 50, 75)

>>> T2=(10,20,30,40)
>>> T2[2:4]*3
(30, 40, 30, 40, 30, 40)
```

▢ Slice Operator:

tuple-name[start:end] will give you elements between indices **start** to **end-1**.

```
>>> alpha=('q','w','e','r','t','y')
```

```
>>> alpha[1:-3]
```

```
('w', 'e')
```

```
>>> alpha[3:65]
```

```
('r', 't', 'y')
```

```
>>> alpha[-1:-5]
```

```
()
```

```
>>> alpha[-5:-1]
```

```
('w', 'e', 'r', 't')
```

List-name[start:end:step] will give you elements between indices **start** to **end-1** with skipping elements as per the value of **step**.

```
>>> alpha[1:5:2]
```

```
('w', 'r')
```

```
>>> alpha[ : : -1]
```

```
('y', 't', 'r', 'e', 'w', 'q')      #reverses the tuple
```

▢ Comparison Operators:

- o Compare two tuples
- o Python internally compares individual elements of tuples in lexicographical order.
- o It compares the each corresponding element must compare equal and two sequences must be of the same type.
- o For **non-equal comparison** as soon as it gets a result in terms of True/False, from corresponding elements' comparison. If Corresponding elements are equal, it goes to the next element and so on, until it finds elements that differ.

Example:

```
>>> T1 = (9, 16, 7)
```

```
>>> T2 = (9, 16, 7)
```

```
>>> T3 = ('9','16','7')
```

```
>>> T1 == T2
```

```
True
```

```
>>> T1==T3
```

```
False
```

```
>>> T4 = (9.0, 16.0, 7.0)
>>> T1==T4
True
>>> T1<T2
False
>>> T1<=T2
True
```

Tuple Methods:

Consider a tuple:

```
subject=("Hindi","English","Maths","Physics")
```

S. No.	Function Name	Description	Example
1	len()	Find the length of a tuple. Syntax: len (tuple-name)	>>>subject=("Hindi","English","Maths", "Physics") >>> len(subject) 4
2	max()	Returns the largest value from a tuple. Syntax: max(tuple-name)	>>> max(subject) 'Physics'
Error: If the tuple contains values of different data types, then it will give an error because mixed data type comparison is not possible. <pre>>>> subject = (15, "English", "Maths", "Physics", 48.2) >>> max(subject)</pre> TypeError: '>' not supported between instances of 'str' and 'int'			
3.	min()	Returns the smallest value from a tuple. Syntax: min(tuple-name)	>>>subject=("Hindi","English","Maths", "Physics") >>> min(subject) 'English'
Error: If the tuple contains values of different data types, then it will give an error because mixed data type comparison is not possible. <pre>>>> subject = (15, "English", "Maths", "Physics", 48.2) >>> min(subject)</pre> TypeError: '>' not supported between instances of 'str' and 'int'			

4	index()	Returns the index of the first element with the specified value. Syntax: tuple-name.index(element)	>>>subject=("Hindi","English","Maths","Physics") >>> subject.index("Maths") 2
5	count()	Return the number of times the value appears. Syntax: tuple-name.count(element)	>>> subject.count("English") 1

Tuple Packing and Unpacking:

Tuple Packing: Creating a tuple from set of values.

Example:

```
>>> T=(45,78,22)
>>> T
(45, 78, 22)
```

Tuple Unpacking : Creating individual values from the elements of tuple.

Example:

```
>>> a, b, c=T
>>> a
45
>>> b
78
>>> c
22
```

Note: Tuple **unpacking** requires that the number of variable on the left side must be equal to the length of the tuple.

Delete a tuple:

The **del** statement is used to delete elements and objects but as you know that tuples are immutable, which also means that individual element of a tuple cannot be deleted.

Example:

```
>> T=(2,4,6,8,10,12,14)
```

```
>>> del T[3]
```

TypeError: 'tuple' object doesn't support item deletion

But you can delete a complete tuple with ***del*** statement as:

Example:

```
>>> T=(2,4,6,8,10,12,14)
```

```
>>> del T
```

```
>>> T
```

NameError: name 'T' is not defined

DICTIONARY IN PYTHON

INTRODUCTION:

- Dictionary is a collection of elements which is **unordered, changeable and indexed**.
- Dictionary has **keys** and **values**.
- Doesn't have index for values. Keys work as indexes.
- Dictionary doesn't have duplicate member means no duplicate key.
- Dictionaries are enclosed by curly braces { }
- The key-value pairs are separated by commas (,)
- A dictionary key can be almost any Python type, but are usually numbers or strings.
- Values can be assigned and accessed using square brackets [].

CREATING A DICTIONARY:

Syntax:

```
dictionary-name = {key1:value, key2:value, key3:value, keyn:value}
```

Example:

```
>>> marks = { "physics" : 75, "Chemistry" : 78, "Maths" : 81, "CS":78 }
```

```
>>> marks
```

```
{'physics': 75, 'Chemistry': 78, 'Maths': 81, 'CS': 78}
```

```
>>> D = { }      #Empty dictionary
```

```
>>> D
```

```
{ }
```

```
>>> marks = { "physics" : 75, "Chemistry" : 78, "Maths" : 81, "CS":78 }
```

```
{'Maths': 81, 'Chemistry': 78, 'Physics': 75, 'CS': 78}
```

there is no guarantee that elements in dictionary can be accessed as per specific order.

Note: Keys of a dictionary must be of immutable types, such as **string, number, tuple**.

Example:

```
>>> D1={ [2,3]: "hello" } TypeError:
```

```
unhashable type: 'list'
```

Creating a dictionary using dict() Constructor:

A. use the dict() constructor with single parentheses:

```
>>> marks=dict(Physics=75,Chemistry=78,Maths=81,CS=78)
```

```
>>> marks
```

```
{'Physics': 75, 'Chemistry': 78, 'Maths': 81, 'CS': 78}
```

In the above case the keys are not enclosed in quotes and equal sign is used for assignment rather than colon.

B. dict() constructor using parentheses and curly braces:

```
>>> marks=dict({"Physics":75,"Chemistry":78,"Maths":81, "CS":78})
```

```
>>> marks
```

```
{'Physics': 75, 'Chemistry': 78, 'Maths': 81, 'CS': 78}
```

C. dict() constructor using keys and values separately:

```
>>> marks=dict(zip(("Physics","Chemistry","Maths","CS"),(75,78,81,78)))
```

```
>>> marks
```

```
{'Physics': 75, 'Chemistry': 78, 'Maths': 81, 'CS': 78}
```

In this case the keys and values are enclosed separately in parentheses and are given as argument to the zip() function. zip() function clubs first key with first value and so on.

D. dict() constructor using key-value pairs separately:

Example-a

```
>>> marks=dict(['Physics',75],['Chemistry',78],['Maths',81],['CS',78])
```

list as argument passed to dict() constructor contains list type elements.

```
>>> marks
```

```
{'Physics': 75, 'Chemistry': 78, 'Maths': 81, 'CS': 78}
```

Example-b

```
>>> marks=dict(('Physics',75),('Chemistry',78),('Maths',81),('CS',78))
```

```
# tuple as argument passed to dict( ) constructor contains list type elements
```

```
>>> marks
```

```
{'Physics': 75, 'Chemistry': 78, 'Maths': 81, 'CS': 78}
```

Example-c

```
>>> marks=dict((( 'Physics',75),('Chemistry',78),('Maths',81),('CS',78)))
```

```
# tuple as argument to dict( ) constructor and contains tuple type elements
```

```
>>> marks
```

```
{'Physics': 75, 'Chemistry': 78, 'Maths': 81, 'CS': 78}
```

ACCESSING ELEMENTS OF A DICTIONARY:

Syntax:

```
dictionary-name[key]
```

Example:

```
>>> marks = { "physics" : 75, "Chemistry" : 78, "Maths" : 81, "CS":78 }
```

```
>>> marks["Maths"]
```

81

```
>>> marks["English"]           #Access a key that doesn't exist causes an error
```

KeyError: 'English'

```
>>> marks.keys( )           #To access all keys in one go
```

```
dict_keys(['physics', 'Chemistry', 'Maths', 'CS'])
```

```
>>> marks.values( )         # To access all values in one go
```

```
dict_values([75, 78, 81, 78])
```

Lookup : A dictionary operation that takes a key and finds the corresponding value, is called lookup.

TRAVERSING A DICTIONARY:

Syntax:

```
for <variable-name> in <dictionary-name>:
```


statement

Example:

```
>>> for i in marks:

    print(i, ": ", marks[i])
```

OUTPUT:

physics : 75

Chemistry : 78

Maths : 81

CS : 78

CHANGE AND ADD THE VALUE IN A DICTIONARY:

Syntax:

dictionary-name[key]=value

Example:

```
>>> marks = { "physics" : 75, "Chemistry" : 78, "Maths" : 81, "CS":78 }
```

```
>>> marks
```

```
{'physics': 75, 'Chemistry': 78, 'Maths': 81, 'CS': 78}
```

```
>>> marks['CS']=84      #Changing a value
```

```
>>> marks
```

```
{'physics': 75, 'Chemistry': 78, 'Maths': 81, 'CS': 84}
```

```
>>> marks['English']=89      # Adding a value
```

```
>>> marks
```

```
{'physics': 75, 'Chemistry': 78, 'Maths': 81, 'CS': 84, 'English': 89}
```

DELETE ELEMENTS FROM A DICTIONARY:

There are **two** methods to delete elements from a dictionary:

(i) using **del** statement

(ii) using **pop()** method

(i) Using **del** statement:

Syntax:

```
del dictionary-name[key]
```

Example:

```
>>> marks
```

```
{'physics': 75, 'Chemistry': 78, 'Maths': 81, 'CS': 84, 'English': 89}
```

```
>>> del marks['English']
```

```
>>> marks
```

```
{'physics': 75, 'Chemistry': 78, 'Maths': 81, 'CS': 84}
```

(ii) Using **pop()** method:

It deletes the key-value pair and returns the value of deleted element.

Syntax:

```
dictionary-name.pop( ) Example:
```

```
>>> marks
```

```
{'physics': 75, 'Chemistry': 78, 'Maths': 81, 'CS': 84}
```

```
>>> marks.pop('Maths')
```

```
81
```

CHECK THE EXISTANCE OF A KEY IN A DICTIONARY:

To check the existence of a key in dictionary, two operators are used:

(i) in : it returns **True** if the given key is present in the dictionary, otherwise **False**.

(ii) not in : it returns **True** if the given key is not present in the dictionary, otherwise **False**.

Example:

```
>>> marks = { "physics" : 75, "Chemistry" : 78, "Maths" : 81, "CS":78 }
```

```
>>> 'Chemistry' in marks
```

```
True
```

```
>>> 'CS' not in marks
```

False

```
>>> 78 in marks    # in and not in only checks the existence of keys not values
```

False

However, if you need to search for a value in dictionary, then you can use `in` operator with the following syntax:

value in dictionary-name.values()

Example:

```
>>> marks = { "physics" : 75, "Chemistry" : 78, "Maths" : 81, "CS":78 }
```

```
>>> 78 in marks.values( ) True
```

DICTIONARY FUNCTIONS:

Consider a dictionary **marks** as follows:

```
>>> marks = { "physics" : 75, "Chemistry" : 78, "Maths" : 81, "CS":78 }
```

S. No.	Function Name	Description	Example
1	len()	Find the length of a dictionary. Syntax: len (dictionary-name)	>>> len(marks) 4
2	clear()	removes all elements from the dictionary Syntax: dictionary-name.clear()	>>> marks.clear() >>> marks { }
3.	get()	Returns value of a key. Syntax: dictionary-name.get(key)	>>> marks.get("physics") 75
Note: When key does not exist it returns no value without any error. >>> marks.get('Hindi') >>>			

4	items()	returns all elements as a sequence of (key,value) tuples in any order. Syntax: dictionary-name.items()	>>> marks.items() dict_items([('physics', 75), ('Chemistry', 78), ('Maths', 81), ('CS', 78)])
Note: You can write a loop having two variables to access key: value pairs. <pre>>>> seq=marks.items() >>> for i, j in seq: print(j, i)</pre> OUTPUT: 75 physics 78 Chemistry 81 Maths 78 CS			
5	keys()	Returns all keys in the form of a list. Syntax: dictionary-name.keys()	>>> marks.keys() dict_keys(['physics', 'Chemistry', 'Maths', 'CS'])
6	values()	Returns all values in the form of a list. Syntax: dictionary-name.values()	>>> marks.values() dict_values([75, 78, 81, 78])
7	update()	Merges two dictionaries. Already present elements are override. Syntax: dictionary1.update(dictionary2)	
Example: <pre>>>> marks1 = { "physics" : 75, "Chemistry" : 78, "Maths" : 81, "CS":78 } >>> marks2 = { "Hindi" : 80, "Chemistry" : 88, "English" : 92 } >>> marks1.update(marks2) >>> marks1 {'physics': 75, 'Chemistry': 88, 'Maths': 81, 'CS': 78, 'Hindi': 80, 'English': 92}</pre>			